Práctica 1.5 — Modelos SQLAlchemy y conexión a BD

Objetivo: persistir datos reales en SQLite.

Enunciado: instala `sqlalchemy`; crea `app/database.py` (o colócalo en el directorio que tengas definido en tu arquitectura) con `engine` (SQLite, `taskflow.db`) y `SessionLocal`; crea `app/models.py`(o colócalo acorde a la arquitectura definida) con `Task`, `Project`, `User` y `Category` (declarative) conforme al documento de especificación del proyecto teniendo en cuenta que todos los usuarios podrán acceder a todos los proyectos; crea la dependencia `get_db()` que provea una sesión por petición.

Criterios de aceptación: `Task` tiene FK a `Project`. La sesión se cierra correctamente (`try/finally` o `yield`).

Entregable: Enlace a GitHub con el proyecto avanzado + `database.py` + `models.py` en Classroom.

Criterio	NL	EP	L	E
Modelos SQLAlchemy	No hay modelos	Modelos sin relación entre ellos	`Task`, `Project` , `Category y Users`con FK y relación `1—N`,	`back_populates` correctos + `cascade` definido
Ciclo de vida de la sesión	No se cierra la sesión	Se cierra pero sin `finally`	`get_db()` con `yield` en `try/finally`	`expire_on_commit=False` y pooling documentado
Configuración del engine	Sin engine o mal configurado	Engine con URL por defecto	`engine` con SQLite y `check_same_thread=False` justificado	`pool_pre_ping`, timeouts y configuración por entorno
Concepto de ORM y relaciones	No sabe qué es un ORM	Distinción parcial	Explica el mapeo clase-tabla y su ventaja	Compara ORM con SQL directo y sabe cuándo usar cada uno
Integración con FastAPI	La sesión no se inyecta	`Depends(get_db)` en algún sitio pero no en todos	Todos los endpoints usan `Depends(get_db)`	Dependencias anidadas (`get_current_user` + `get_db`)
Documentación	Sin comentarios	Comentarios parciales	Comentarios en modelos y database	Diagrama del esquema de BD documentado